Stock Movement Prediction from Price Trends and News Sentiment

Team: Honoré Alexander, Owen Holt, Pranavi Khanna, Dylan Lim, Yihong Li, Ethan Lee, Dan Firstenberg, Hyeongseung Nam, Oscar Pineda, Kevin Zhang, Zachary Chan, Vicente Aguayo

November 20th, 2025

Abstract

Human analysts and investors are known to struggle in predicting stock behaviour accurately due to many factors such as cognitive biases, emotional decision making, and complex macroeconomic trends. The central aim of our project is to improve next-day stock movement prediction by leveraging machine learning models to analyze historical price trends alongside macroeconomic news sentiment. To address this topic, we constructed a data management process linking historical company-specific news headlines to corresponding stock performance data across 369 individual tickers. Using this data we developed a Multi-Layer Perceptron (MLP) neural network which performs feature level fusion, combining encoded news sentiment with normalized stock price indicators to output a probability of next-day positive stock movement using a binary classification framework. The results of our model suggest that it is possible to use macroeconomic news sentiment and price trends to improve prediction accuracy of next-day stock movement. Our model achieved an accuracy comparable to models that only utilized news sentiment while employing a significantly less complex machine learning architecture. Our approach shines light on macroeconomic news sentiment and stock movement features as a potentially effective combination of features to predict stock prices.

Keywords: Machine Learning, Exchange-Traded Funds(ETF's), Batch Gradient Descent, Ordinary Least Squares

1 Introduction

Machine learning has become a staple in the field of quantitative finance for predicting stock prices and market trends. These models outperform traditional statistical techniques by detecting and representing complex patterns in large datasets [Malanin]. One of the biggest challenges is training a model that can accurately predict stock prices, given the unpredictability of the stock market. Obvious approaches, such as using historical stock prices as the training dataset, have been attempted but these endeavours are often difficult due to factors such as dataset noise which makes data preprocessing very difficult. Naturally, stock prices and market prices are also often affected by other factors such as unexpected economic or social events, and investors' psychological behaviors [Usmani and Shamsi]. News-based modeling introduces additional complexity. Headlines vary widely in tone, publication timing, and relevance, and their relationship to stock movement is highly nonlinear. For example, a model based on a Long Short-Term Memory (LSTM) architecture trained in 2023 attempting to predict individual stock prices based on stock-related news headlines surpassed baseline models to achieve an accuracy of around 51%[Chen]. While this accuracy level is only slightly better than a coin flip, this performance level is still significant especially with regards to stock prediction. Many modern machine learning models rely heavily on technical indicators such as price, moving averages, volume, and more, but disregard or fail to incorporate vital macroeconomic signals such as geopolitical events, inflation data, and macroeconomic news sentiment. What's currently missing is a structured implementation of macroeconomic and company-specific news signals to current traditional stock market features. In this paper, we address this gap by developing a machine learning network that integrates both macroeconomic news sentiment with traditional trend indicators for next-day stock movement prediction. Using our preprocessed dataset we compute a set of seven commonly used stock features - including 1-day return, log trading volume, short-term momentum (3-day and 7-day), recent volatility, price position within the daily range, and volume trend - to provide context of a stock's behavior alongside news sentiment. These features are then fused with high-dimensional news embeddings to capture interactions between market dynamics and macroeconomic sentiment. Research shows that AI is most impactful when used to interpret diverse data rather than automate tasks. Its strength lies in improving prediction, decision making and product innovation which helps firms grow and increase valuation [Babina]. Our model follows this principle by combining sentiment and price based features to extract more meaningful patterns for stock movement prediction. In addition, recent financial research indicates that models trained purely on historical price data struggle when markets enter new or unstable times, particularly during high-volatility or sentiment-driven periods such as early COVID-19. This motivates using sentiment driven features alongside technical indicators to improve robustness across shifting market conditions, rather than relying only on past price patterns [Begenau]. We realize that due to the inherent complexities of this topic and limits on time and resources, we may not be able to make breakthroughs in model performance. Rather than creating a competitive model, we aim to produce a model that is simple, lightweight, and comparable in accuracy to models such as those defined in [Chen]. Our goal is to show that the specific interaction of features, specifically news headline sentiment and technical stock/stock movement fea-

tures, has significant predictive value even when trained on less complex machine learning architectures. Our final trained model demonstrates the performance of this specific feature combination, yielding an accuracy value similar to the LSTM model in [Chen] but on a Multi-Layer Perceptron (MLP), a simpler and less computationally expensive architecture.

2 Methods

To train our model, we followed a conventional machine learning development model.

2.1 Data Sourcing and Preprocessing

2.1.1 Dataset Identification

In order to train a predictive model based on stock prices, we required at least two datasets: one containing daily stock price and performance across a large span of time, and another dataset containing company or sector-related news corresponding to the same timeframe of the stock price data. We chose the years 2018 to 2023 as our ideal timeframe for training data, as it represents a time that is neither too old nor recent. This timeframe encapsulates multiple market regimes, including the bearish COVID-19 market, and the sharp rebound shortly after. Our reasoning is that having a variety of regimes would give our model a higher predictive power and better equip it to identify both long and short opportunities with less skew. Our main stock information dataset was extracted from Yahoo Finance, where we pulled daily individual stock data from the years 2018 to 2023. The dataset contains multiple meaningful attributes, including dates, opens, highs, lows, closes, volume, dividends, stock splits, and companies. Our other datasets contained data on news headlines, all sourced from Kaggle. They featured a combination of both broader world news headlines and company or sector specific news headlines.

2.1.2 Dataset Merging

Our aim here was to merge stock information and news headlines from their respective datasets into a unified dataset. Using the NumPy library, we were able to represent our data as a dataframe type that we could easily iterate over. We first performed basic data cleaning steps, including date conversions, standardizing data types, and removing missing data. Our main task was to assign each entry in the stock information data with the corresponding news headlines for that day and specific stock ticker. We utilized a keyword-mapping approach commonly used in Natural Language Processing. For each ticker, we produced related keywords that could be used to identify related news headlines, i.e. "AAPL", "Apple", "Apple Inc", "iPhone", "MacBook" correspond to Apple (AAPL). Using a map to represent these keywords, we iterated through the datasets to create these stock-news headline associations. Our first iteration was on the world news dataset, yielding about 1600 successful associations. For our second iteration with the dataset containing company-specific headlines, we were able to increase the total to just above 38,000 associations.

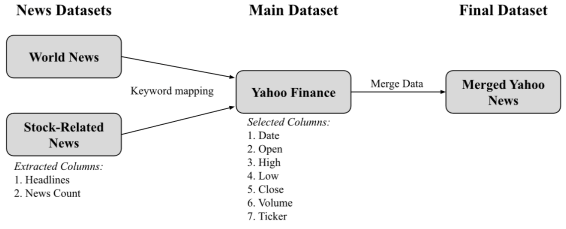

Figure 1: Data Flow

2.1.3 Exploratory Data Analysis

Our goal with the exploratory data analysis was to better understand the data's overall behavior. Specifically, we focused on the trends of the news headlines. We wanted to understand how the news head-

lines were distributed, and how they affected the stock prices.
We discovered that the news headlines were heavily concentrated in the years 2019-2020. Using a line plot of the top-10 tickers, we visualize this below.

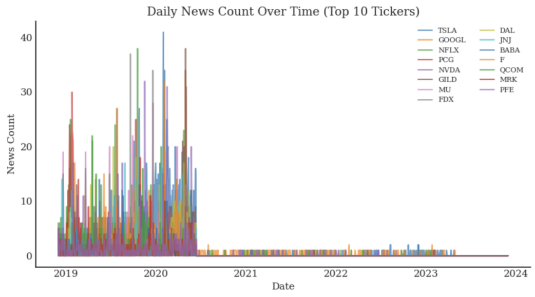

Figure 2: Period Distribution of Data

Additionally, we discovered that there was no significant relationship between a stock's daily price change and its number of news headlines. We visualize this in the scatterplot below. Given this, it was clear that our machine learning model would have to use more intricate features than just the number of news headlines. Interpreting the text of the headlines would be a necessary step.

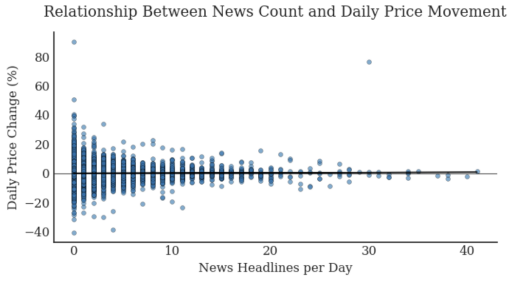

Figure 3: Daily Price vs. # Headlines

2.1.4 Label Generation
Our primary goal was to create a binary target variable (y) that captures the single most important piece of information: **Will the stock close higher tomorrow?**

To answer that question, we first need tomorrow's price. We used the Pandas to calculate the next day's simple return (R_{t+1}) for each stock individually. This is key because it aligns the closing price from day $t+1$ (which is the next day) with the features from day t, which prevents lookahead bias, which was one of our biggest risks. Lookahead bias is when we use future correlations to predict future prices, which gives us a falsely accurate result. This simple return value was only visible to the training data.

$$R_{t+1} = \frac{\text{Close}_{t+1}}{\text{Close}_t} - 1 = \text{future\_return\_1d}$$

Binary Label Creation

Once we created the return, R_{t+1}, we converted it into our binary classification target (Label_t). This label is what we want our model to learn how to predict.

$$\text{Label}_t = \begin{cases} 1 & \text{if } R_{t+1} > 0 \quad \text{(Stock Up)} \\ 0 & \text{if } R_{t+1} \leq 0 \quad \text{(Stock Down)} \end{cases}$$

2.1.5 Label Generation
We calculated a set of seven standard and frequently used technical features (p) to give our model context about the stock's recent behavior, not just the news. That way, we can combine both the stock's price movement and our news analysis to try to make an accurate prediction.

1. **1-Day Close Return** ($R_{1d,t}$) This is the simplest way for us to determine a stocks current momentum. It only tells the model if the stock has been rising or falling today.

$$R_{1d,t} = \frac{\text{Close}_t - \text{Close}_{t-1}}{\text{Close}_{t-1}}$$

2. **Log Volume** (vol_log_t) We also wanted to measure the volume of shares being traded. We used the natural logarithm of volume to compress this scale, making it easier for the neural network to handle especially if the

volume is very large (can be in the millions).

$$\text{vol\_log}_t = \ln(1 + \text{Volume}_t)$$

3. **Momentum Features** ($R_{kd,t}$) These features calculated over $k = 3$ and $k = 7$ days, represent short term trends for our model. Similar to 50 and 200-SMA, but for a shorter time period. If a stock is consistently trending, these values will be high, and we hoped this would indicate a continuation of it's current trend.

$$R_{kd,t} = \frac{\text{Close}_t - \text{Close}_{t-k}}{\text{Close}_{t-k}}, \quad k \in \{3, 7\}$$

4. **7-Day Volatility** ($\text{volatility}_{7d,t}$) This feature tells the model how volatile the stock has been lately. It's the standard deviation (Std) of the daily returns over the past week. High volatility usually signals "choppiness" or uncertainty in how the market is moving.

$$\text{volatility}_{7d,t} = \text{Std}(\{R_{1d,t-6}, \ldots, R_{1d,t}\})$$

5. **Price Position** (price_position_t) This is a normalized value that shows where the closing price sits relative to the daily high and low. A value near 1 means the stock closed near its high, which can signal buying pressure because it means price went up, and kept going. On the other hand, closing near the low means that sellers were dominating the market because price kept moving downwards . We also used ϵ to prevent an error that was caused by dividing by zero.

$$\text{price\_position}_t = \frac{\text{Close}_t - \text{Low}_t}{\text{High}_t - \text{Low}_t + \epsilon}, \quad \epsilon = 10^{-8}$$

6. **Volume Trend** (volume_trend_t) This metric tells us if today's trading activity was unusually high or low compared to the 7-day average. Extreme volume can confirm price moves or signal reversals.

$$\text{volume\_trend}_t = \frac{\text{Volume}_t}{\text{Avg}(\text{Volume}_{t-6}, \ldots, \text{Volume}_t)} - 1$$

2.1.6 Sentiment Feature Generation

We knew price alone wasn't enough; we needed to capture the qualitative factor of news sentiment. The embedding.py script handled this by converting news headlines into a powerful numerical vector e.

We chose the specialized **FinBERT** model because it was pre-trained on financial text, giving it a much better understanding of phrases and terms like "acquisition" or "earnings beat" than general models. To get a single vector $\mathbf{e} \in \mathbb{R}^{768}$ that represents the headline's overall sentiment, we performed an Attention-Masked Mean Pooling, which took our sequence of "words" and transformed it into a single numerical vector. This made it a lot easier for us to handle the data in the context of our model.

The below formula calculates the weighted average of all the token vectors ($\mathbf{H}_i$) in the headline, where the weight ($\mathbf{M}_i$) makes sure we only count real words and ignore any empty "padding" tokens. Padding tokens are added to make all strings the same lengths. Instead of averaging all of our vectors (which, because of the 0 passing, would skew our output downwards), we only sum the vectors that correspond to real words by multiplying them by the mask (and thus ignoring those that are multiplied by 0).

$$\mathbf{e} = \frac{\sum_{i=1}^{L} \mathbf{H}_i \cdot \mathbf{M}_i}{\sum_{i=1}^{L} \mathbf{M}_i}$$

- $\mathbf{H}_i$: The vector output for token i. This is essentially the raw information.
- $\mathbf{M}_i$: The attention mask (1 for token, 0 for padding). This is the vector of 1s and 0s made during tokenization.

2.2 Model Data Preparation and Normalization

We also recognized that our price features $\mathbf{p}$ were wildly different (e.g., $R_{1d,t}$ is small, while vol_log_t is large). This often makes neural networks unstable because nodes with values of different scales end up being analyzed the same. Therefore, we standardized them using Z-score normalization. This process gives every feature a mean of 0 and a standard deviation of 1.

$$\mathbf{p}_{\text{norm}} = \frac{\mathbf{p} - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

We only computed the mean (μ_{train}) and standard deviation (σ_{train}) from the training data and applied them to all sets. This was to make sure that we can keep and maintain the integrity of our test data.

2.3 Model Architecture

2.3.1 Model Definition

The primary piece of our project is the creation of our NewsClassifier network which we implemented as a *Multi-Layer Perceptron (MLP)* configured for binary classification. The objective of our network is to predict the direction of a given stock's closing price on the following trading day.

2.3.2 Input Fusion

In order to allow our model to learn from both textual (news) and numerical (price) data, we applied a feature-level fusion strategy allowing our model to learn more effectively by fusing features in an efficient way. More specifically, we concatenated a 786-dimensional news embedding vector (e) with a 7-dimensional vector of normalized price features, p_{norm}, to form a single vector:

$$\mathbf{x} = [\mathbf{e}, \mathbf{p}_{\text{norm}}]$$

This unified input allows our model to capture interactions between fundamental signals derived from investor sentiment to macroeconomic news and technical market indicators derived from historical stock price data.

2.3.3 Layers and Output

The formed vector x is passed through the network's several hidden layers with non-linear ReLU activation to learn complex interactions between sentiment and price signals. The final output layer consists of a single neuron which is passed through the Sigmoid function (σ) to produce a probability $y \in [0, 1]$ representing the likelihood that the stock price will increase on the next trading day.

$$\hat{y} = \sigma(\mathbf{W}_3\mathbf{h}_2 + \mathbf{b}_3), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

2.4 Model Training

2.4.1 Loss Function

For our loss function we used the Binary Cross-Entropy Loss (L) that we learned in class. It heavily penalizes our model when its predicted probability ($\hat{y}$) is far from the true label (y).

2.4.2 Update Rule

The Adam update rule is what gave us a lot of speed and success in the optimization section. It uses momentum ($\hat{\mathbf{m}}_t$) to encourage convergence in consistent directions, and adaptive scaling ($\hat{\mathbf{v}}_t$) to give each parameter its own custom learning rate. Custom learning rates is crucial here especially for variables of different magnitudes, and the nature of Adam is more convex than gradient descent is which makes it less likely for our model to get stuck in a local minima.

$$\text{Parameters}_{t+1} = \text{Parameters}_t - \frac{\eta}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \cdot \hat{\mathbf{m}}_t$$

2.4.3 Epochs and Batches

Because of the nature of our prediction values, it isn't sufficient to do only a few passes of our network. Instead, we split our data into mini-batches of size $2\hat{6}$ (64, which is large enough to take advantage

of our GPU processing but not too large so as to prevent us from using our massive feature input $\mathbf{x} = [\mathbf{e}, \mathbf{p}_{\text{norm}}]$), and ran these sequentially through 10 different epochs.

2.4.3 Epochs and Batches

Finally, to evaluate our success, we measured our performance using accuracy. We converted the probability output $\hat{y}$ into a hard prediction $\hat{y}_{\text{pred}}$ using a 0.5 threshold:

$$\hat{y}_{\text{pred}} = \begin{cases} 1 & \text{if } \hat{y} > 0.5 \\ 0 & \text{if } \hat{y} \leq 0.5 \end{cases}$$

If our model was more than 50% confident that price would go up the next day, we allowed it to make that prediction. We modeled our accuracy as the number of correct predictions over the total number of samples. An accuracy of about 50% would signal to us that we have some sort of edge in the market.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Samples}}$$

3 Results

This is the resulting accuracy, loss, and ROC of our model. By the end of 10 epochs, our training accuracy reached around just above 53% accuracy. Our Area Under Curve (AUC) for our Receiver Operating Characteristic (ROC) is 0.550.

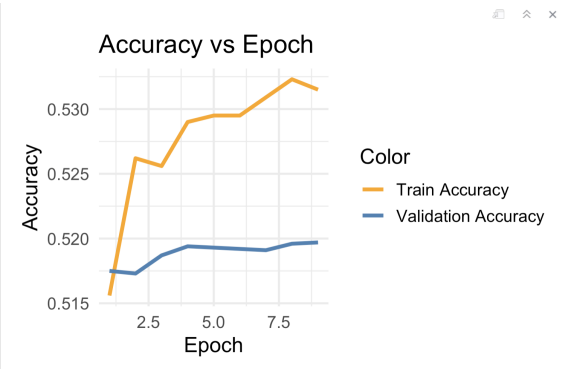

Figure 4: Accuracy

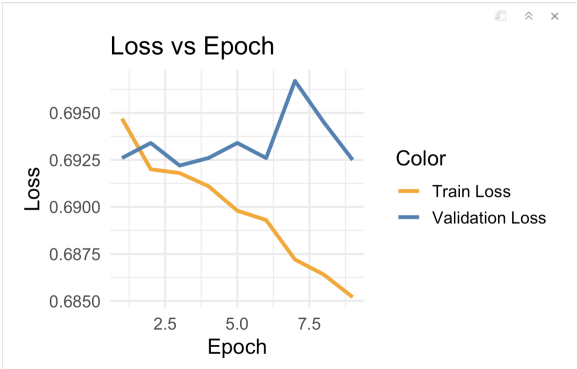

Figure 5: Loss

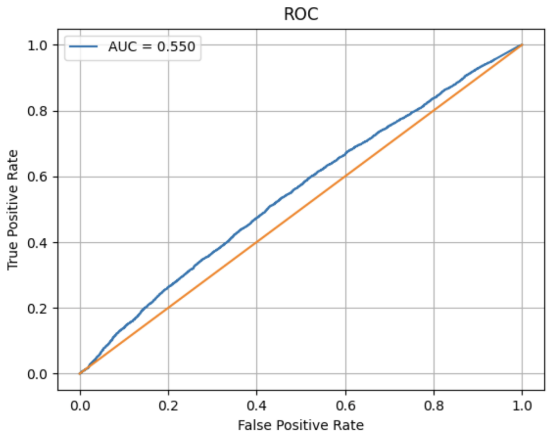

Figure 6: Receiver Operating Characteristics

4 Discussion

4.1 Data and Preprocessing

4.1.1 Data Sourcing

While we had quite a substantial amount of stock-news headlines associations (rows) in our dataset, there were still quite a few limitations in our dataset identification and preprocessing steps that may have affected our results.

For the datasets themselves, there could have been sourcing biases especially with regards to news coverage. Datasets from Kaggle may not have the most comprehensive coverage, potentially missing news from sources such as local journals or live newswire services. In addition, larger, more established companies are more often the subject of headlines, causing an imbalance in coverage that may cause

the model to consider these stocks more heavily during training.

4.1.2 Dataset Merging

As for the data merging step, our main concern had to do with our keyword-based approach in merging news headlines data into our stock price dataset. Our keyword mapping was created manually, which introduced both false positives (irrelevant news headlines containing keywords) and false negatives (relevant news headlines not containing keywords). This problem was the most apparent when merging news headlines from the world news dataset, and less so when merging from the stock-related news dataset.

One major limitation highlighted in similar financial prediction studies is that proxy-based matching (such as keywords to associate news with companies) can lead to systematic errors. When demographic or classification variables are predicted using imperfect proxy data, apparent relationships become distorted, often underestimating real predictive disparities by over 40 percent [Greenwald]. Similarly our keyword based matching likely introduced both false positives and false negatives, reducing reliability of or sentiment associations.

Other potential issues included coverage and news context. When merging news headlines with stocks, we achieved a 75.2% coverage, associating 369 out of the 491 stock tickers with news headlines. About one-fourth of the stocks in the dataset had no news coverage at all, introducing a selection bias that limits our model to stocks for companies that are larger and mentioned more often in the news. In addition, the majority of news headlines turned out to be concentrated within the years 2019 and 2020. This introduces a bias within the dataset, as well as being from an unstable time period (COVID-19 pandemic) that very likely affects predictive power.

4.1.3 Labeling and Technical Feature Extraction

The main issue driving our label creation was bias. As machine learning models are deployed in countless sectors, many of which where errors could be fatal, one of the most critical sources of error to be aware of are biases in data [Weijia]. Our main concern was lookahead bias, which is when we use future correlations to predict future prices, which gives us a falsely accurate result. We had to make sure we only use information available to us "today" to make predictions for "tomorrow". Therefore, we had to align the closing price from day t + 1 to features from day t (see 2.1.5).

We decided that merely using stock price information and news headlines would not be sufficient for training and accurate models. It's emphasized that both public and private (company internal/insider trading) information related to stocks matter significantly when it comes to modeling stock price and speculation [Saberironaghi]. As we do not have access to private information, we instead made do with seven context features calculated from publicly available market data (see 2.1.5). This additional context about stock movement was crucial in our training.

4.1.4 Sentiment Feature Generation

When conducting sentiment analysis on news headlines, we initially approached it very generally, associating good sentiment with conventionally positive word connotations, and bad sentiment with conventionally negative word connotations. However, we quickly realized that this approach would miss domain-specific terms ("acquisition", "earnings beat", etc.) and could misidentify sentiment direction for certain terms (i.e. positive

sentiment for "higher [costs]").

We instead decided to utilize the specialized FinBERT model, which was already pre-trained on financial texts, giving it a much better understanding of finance-related terms than general models. We additionally performed Attention-Masked Mean Pooling to improve the weights slightly by removing irrelevant (masked) token vector details.

Bias in machine learning does not only originate from data collection but can also be reinforced during feature interpretation and model training especially when sentiment is extracted using generalized language models [Mehrabi]. General sentiment tools may misinterpret domain-specific terms such as "beat earnings" or "market risk," which can lead to distorted sentiment scores. This further supports our use of FinBERT, which is trained specifically on financial text and is better aligned with the language used in news related stock movement.

4.2 Model Architecture

4.2.1 Input Fusion and Model Definition

Our completed input vector was a concatenation of our 768-dimensional news embedding vector (e) and our 7-dimensional normalized price features vector (p). We decided to implement a Multi-Layer Perceptron due to its simplicity as well as the fact that our data is already a fixed-size input. Other architectures we considered included Recurrent Neural Networks (RNN), particularly Long Short-Term Memory (LSTM), and transformers. These were quickly excluded from consideration as we decided we would not model any sequences or time-series, which these architectures excelled at, at the cost of computational resource usage [Suresh, Harini, and John Guttag.].

4.2.2 Layers and Output

In our layers, we decided to use mainly ReLU activation functions, due to their property of avoiding vanishing gradients and cheap computational cost. Using Sigmoid here would leave us at risk of seeing a vanishing gradient, and converging our derivatives towards zero during backpropagation. However, the output layer's main task is to produce a value that can easily be converted to an interpretable probability. Therefore, since Sigmoid compresses vectors into values between 0 and 1, we passed our final layer through the Sigmoid function (σ).

4.3 Model Training

4.3.1 Gradient Descent

Initially, the model was trained using standard Gradient Descent (GD), as it was one of the primary techniques covered in class. However, this approach led to slow convergence and low validation accuracy. The fixed learning rate struggled with the high-dimensional, noisy financial data, and it could not adapt to the features that needed smaller steps versus those that needed larger ones.

4.3.2 The Switch to Adam

Initially, we attempted to use standard Gradient Descent (GD), since it is the most standard optimization technique we learned in class. However, we quickly found our validation accuracy to be low and training was unbearably slow. Our fixed learning rate was struggling with the high-dimensional, noisy financial data, and it couldn't adapt to the features that needed smaller steps versus those that needed bigger ones

After researching other options, we decided to implement Adam (Adaptive Moment Estimation), since it builds upon GD but dynamically adjusts the learning rate for every parameter. For a parameter such as vol_log_t that changes quickly and therefore has a really large gradient, $\hat{\mathbf{v}}_t$

would be very large. Therefore, Adam results in a smaller step size than standard Gradient Descent would, which prevents overshooting or infinite loops. Conversely, when $\hat{\mathbf{v}}_t$ is low, step size increases which encourages faster learning. This adaptive approach greatly improved our convergence speed and final accuracy. Rather than taking nearly half an hour to run, our optimization could be done in around a minute.

4.3.3 Loss Function
As we mentioned, our utilization of Binary Cross-Entropy Loss over Mean Squared Error as the loss function was mainly due to the nature of our model. BCE excels at measuring probabilities, which is essential in classification tasks. Additionally, BCE tends to be more convex, meaning that the chance of getting stuck in a local minimum is significantly lower.

4.4 Model Results

4.4.1 Accuracy
Overall, our model results were satisfactory. We achieved a final accuracy of just above 53%. In terms of the problem, this accuracy value is actually a bit higher than the baseline LSTM model in [Usmani]. However, we utilized the Multi-Layer Perceptron architecture, which was simpler compared to an LSTM. The additional difference between our model and the baseline was our addition of technical features of each individual stock, which allowed us to achieve almost equal performance to the LSTM model on an MLP.

4.4.2 Evaluation (ROC)
Our AUC score was 0.550 for our ROC, which suggests that our predictions are only slightly more accurate than a coin flip. However, 0.550 is still above a random baseline of 0.5, suggesting we have a very slight edge in prediction. Considering the nature of the problem in the financial prediction field and the comparable performance to the baseline model from [Usmani and Shamsi], we believe this score is quite satisfactory.

4.4.3 Potential Issues and Disclaimers
Due to the additional numerous potential concerns that we discussed in-depth about the datasets, preprocessing, and model training, our model may not be perfect. The results we achieved depend heavily on our datasets, processing, and methods, and the mentioned limitations may prevent any concrete conclusions to be made from this report and project.

5 Conclusion

The aim of this report was to investigate whether integrating macroeconomic news sentiment with traditional stock price indicators could improve next-day stock movement prediction. More specifically, we wanted to address the gap in existing approaches by implementing macroeconomic and company-specific news signals to current traditional stock market features using a feature-level fusion model that combines news embeddings with seven commonly used stock features in a Multi-Layer Perceptron (MLP) classifier. Through our approach, we managed to achieve a final prediction accuracy of slightly above 53% and an AUC score of 0.550 across 369 stock tickers using preprocessed data from 2018-2023, demonstrating a performance comparable to more complex models while using significantly less computational power and resources. Our results confirm that it is possible to capture interactions between news sentiment and price dynamics using structured feature fusion. Even still, several obstacles and opportunities remain in the field of quantitative finance. Future research should focus on alternatives to keyword-based matching in order to expand on news coverage by possibly using named-entity recognition

or entity-level linking network techniques to aid in reducing misclassification of relevant macroeconomic headlines. From the data portion, using datasets with alternative data information such as call earnings, media sentiment, and advanced macroeconomic indicators would likely aid in improving the robustness of the model.

In the broader scope of stock analysis and quantitative financial machine learning, our work contributes to the growing area of research by demonstrating that diversification of signal integration across news sentiment and market indicators can improve current prediction frameworks for many financial institutions. By showing that our simpler model can perform competitively by carefully structuring feature design and data fusion, this report makes an advancement in the efficiency and robustness of predictive models in predicting market behavior.

References

1. Babina, Tania, et al. “Artificial Intelligence, Firm Growth, and Product Innovation.” ScienceDirect, Jan. 2024.

2. Begenau, Juliane, et al. “Big Data in Finance and the Growth of Large Firms.” ScienceDirect, Aug. 2018.

3. Chen, Qi, et al. “Price Informativeness and Investment Sensitivity to Stock Price.” Finance.Wharton.Upenn.Edu, July 2006.

4. Greenwald, Daniel, et al. “Regulatory Arbitrage or Random Errors? Implications of Race Prediction Algorithms in Fair Lending Analysis.” ScienceDirect, July 2024.

5. Malanin, V. “Comparative Study of Neural Network Architectures in Medical Survival Analysis: Evaluation of MLP, CNN, and LSTM Models.” International Journal of Neural Networks and Deep Learning, vol. 2, no. 2, 2025, pp. 24–43.

6. Mehrabi, Ninareh, et al. “A Survey on Bias and Fairness in Machine Learning.” ACM Computing Surveys, July 2021.

7. Ren, J., et al. “Stock Market Prediction Using Machine Learning and Deep Learning Techniques: A Review.” AppliedMath, vol. 5, 2025, p. 76.

8. Suresh, Harini, and John Guttag. “A Framework for Understanding Sources of Harm Throughout.”

9. Usmani, Shazia, and Jawwad A. Shamsi. “LSTM Based Stock Prediction Using Weighted and Categorized Financial News.” PloS One, vol. 18, no. 3, 7 Mar. 2023.

10. Weijia (Vivian) Li, and Kara M. Kockelman. “How Does Machine Learning Compare to Conventional Econometrics for Transport Data Sets? A Test of ML versus MLE.” Growth and Change, vol. 53, no. 1, Mar. 2022, pp. 342–376.

Author Contributions

- **Honoré Alexander**: Led backend development of neural network system, handling embeddings, deriving price features, and optimizing model to predict if the stock will close higher or lower.
- **Owen Holt**: Developed frontend and model integration, initialized proposal/conceptualized project direction, finalized frontend and UI.
- **Pranavi Khanna**: Researched sources for the report, contributed to writing/editing the report.
- **Dylan Lim**: Created application and backend for model inference, designed the initial frontend. Other contributions to source research, technical report documentation, and slide deck.
- **Yihong Li**: Researched sources and drafted most of the report sections, coordinated report with all teams, formatted and converted report to LaTeX, set up and designed frontend for model inference, contributed to slide deck.
- **Ethan Lee**: Project manager, drafted and noted evidence from all resources used within the report, segmented team responsibilities, ensured all members were being held accountable and responding in communication channels, drafted final presentation slide deck, calculated final data visualization graphs using Python.
- **Dan Firstenberg**: Helped with the development of our neural network by creating a framework, and identifying which technologies we should use.
- **Hyeongseung Nam**: Helped developing frontend.
- **Oscar Pineda**: Researched sources for the report as potential sources, contributed to writing/editing the report.
- **Kevin Zhang**: Researched papers as potential sources, contributed to writing/proofreading the report.
- **Zachary Chan**: Led data cleaning & data preprocessing, created visualizations for data preprocessing & exploratory data analysis, contributed to writing exploratory data analysis section of report.
- **Vicente Aguayo**: Assisted with data cleaning and preprocessing, combined my dataset with Zachery's CSV, processed large portions of merged data, and wrote documentation describing the combined files, schema, and dataset statistics.